Advanced Day 9 — Session 9 (Hours 33–36)

Python Programming (Advanced) • REST APIs and Flask Foundations

Session 9 Overview

Topics Covered Today

- Hour 33 — REST fundamentals + Flask app setup
- Hour 34 — CRUD endpoints for the main resource
- Hour 35 — Serialization + validation (manual, consistent)
- Hour 36 — App structure: blueprints and dependency wiring

Capstone Through-Line

- Expose the existing tracker logic through a Flask API
- Keep the service layer as the center of truth
- Build for maintainability, not framework novelty

Session 9 Outcomes

By the end of today, learners will be able to:

- Create a Flask application with a health endpoint and JSON error contract
- Implement GET, POST, PUT, and DELETE for a tracker resource
- Centralize serialization and validation outside route functions
- Refactor a single-file Flask app into an application factory with explicit wiring

Source Alignment

- `Advanced/lessons/lecture/Day9_Hour1_Advanced.md` → `## Learning Outcomes`
- `Advanced/lessons/lecture/Day9_Hour2_Advanced.md` → `## Learning Outcomes`
- `Advanced/lessons/lecture/Day9_Hour3_Advanced.md` → `## Learning Outcomes`
- `Advanced/lessons/lecture/Day9_Hour4_Advanced.md` → `## Learning Outcomes`

Scope Guardrails for Today

In Scope

- Flask route handlers as thin HTTP adapters
- REST resource naming and HTTP status code conventions
- JSON error envelopes with a consistent shape
- Manual serialization and field-level validation helpers
- Application factory pattern and explicit dependency wiring

Not Yet

- Production deployment, reverse proxies, or WSGI configuration
- OAuth, JWT, or session-based authentication
- API versioning or rate limiting

Hour 33 — REST Fundamentals + Flask App Setup

Source Alignment

`Advanced/lessons/lecture/Day9_Hour1_Advanced.md` → `# Day 9, Hour 1`

Hour 33 Learning Outcomes

By the end of this hour, learners will be able to:

- Create a small Flask application with a reliable health endpoint
- Explain resources, HTTP verbs, status codes, and JSON response contracts
- Use one consistent JSON error shape for every sad path

Source Alignment

`Advanced/lessons/lecture/Day9_Hour1_Advanced.md` → `## Learning Outcomes`

Why an API Layer Now

The capstone tracker already has a working model, service, and repository.

Flask adds a **new interface** — not a new project.

```
Client ──► Flask route ──► TrackerService ──► SQLiteRepository ──► SQLite
```

- Other tools (tests, GUIs, scripts) can call the same logic over HTTP
- Strong layering keeps the service reusable outside Flask
- An API contract makes the next developer's job predictable

> "Routes translate HTTP details into service calls, not contain all business rules."
>
> — `Advanced/lessons/lecture/Day9_Hour1_Advanced.md` → `### Talk points`

REST in Plain English

Term	Meaning
Resource	The thing the API exposes (e.g., a tracker record)
Route	URL pattern mapped to Python code
HTTP method	`GET`, `POST`, `PUT`, `DELETE` — the verb
Status code	Fast numeric signal of what happened
JSON contract	The agreed shape of every request and response

Key Status Codes for Day 9

Code	Meaning
`200`	Success
`201`	Resource created

Demo: Flask App Factory

```python
from __future__ import annotations

from uuid import uuid4
from flask import Flask, jsonify


def error_response(code: str, message: str, status: int):
    payload = {
        "error": {
            "code": code,
            "message": message,
            "request_id": str(uuid4()),
        }
    }
    return jsonify(payload), status


def create_app() -> Flask:
    app = Flask(__name__)

    @app.get("/health")
    def health():
        return jsonify({"status": "ok", "service": "tracker-api"})

    @app.errorhandler(404)
    def not_found(_error):
        return error_response("not_found", "Route not found", 404)

    return app
```

The Health Endpoint

Why `/health` is the right first route:

- Proves the process is up before testing deeper behavior
- Returns a predictable JSON body — no HTML, no guesswork
- Cheapest possible integration smoke test
- Establishes the `create_app()` factory pattern from the start

```
@app.get("/health")
def health():
    return jsonify({"status": "ok", "service": "tracker-api"})
```

Teaching point: Start with one honest route before adding CRUD.

Consistent JSON Error Contract

Every sad path must return the same envelope — no surprises, no HTML tracebacks.

```
{
  "error": {
    "code": "not_found",
    "message": "Record 12 was not found",
    "request_id": "a1b2c3d4-..."
  }
}
```

- `code` — machine-readable identifier for the failure type
- `message` — human-readable explanation
- `request_id` — tracing handle (UUID generated per request)

> "Every error should return JSON, never a surprise HTML traceback."
>
> — `Advanced/lessons/lecture/Day9_Hour1_Advanced.md` → `### Talk points`

Demo Flow: Hour 33

1. Show project folders: `api/`, `src/`, `tests/`, `data/`, `reports/`
2. Create `create_app()` returning a Flask instance
3. Add `GET /health` → `{"status": "ok", "service": "tracker-api"}`
4. Add `error_response(code, message, status)` helper with `request_id`
5. Register a `404` error handler using `error_response`
6. Run `flask --app api.app run --debug`
7. Test `/health` in browser or with `curl`
8. Call a missing route — observe the JSON error, not an HTML traceback

Source Alignment

`Advanced/lessons/lecture/Day9_Hour1_Advanced.md` → `### Demo steps`

Lab: Flask Starter (Hour 33)

Time: 25–35 minutes

Tasks

1. Create `api/app.py` with `create_app()`
2. Add `GET /health` returning `{"status": "ok"}`
3. Implement `error_response` with `code`, `message`, `request_id`
4. Register a global 404 handler
5. Run the server and verify one happy path and one sad path

Completion Criteria

- `/health` returns JSON with `status ok`
- At least one error path returns the shared error envelope

Common Pitfalls — Hour 33

- **Port already in use** — choose a different port rather than restarting the whole lab
- **Debug mode leaks tracebacks** — helpful locally; inappropriate as an API contract
- **Returning plain strings** — makes clients harder to write; always use `jsonify`
- **One giant file** — mixing setup, routing, and service logic increases debugging cost

Source Alignment

`Advanced/lessons/lecture/Day9_Hour1_Advanced.md` → `### Common pitfalls to watch for`

Quick Check — Hour 33

Exit questions:

- What HTTP status code fits a successful create, and why is it not just `200`?
- What happy path did you prove this hour?
- What sad path did you test or plan to test next?
- Which file or module is most important for the next hour?

Hour 34 — CRUD Endpoints for the Main Resource

Source Alignment

`Advanced/lessons/lecture/Day9_Hour2_Advanced.md` → `# Day 9, Hour 2`

Hour 34 Learning Outcomes

By the end of this hour, learners will be able to:

- Implement GET, POST, PUT, and DELETE for the tracker record resource
- Map service-layer outcomes to correct HTTP status codes
- Demonstrate both successful operations and sad paths such as bad JSON and missing records

Source Alignment

`Advanced/lessons/lecture/Day9_Hour2_Advanced.md` → `## Learning Outcomes`

Thin Routes Principle

Routes are **adapters**, not business logic containers.

```
HTTP request
  → parse input
    → call service method
      → serialize output
        → return HTTP response
```

- `400` means the **request** is malformed
- `404` means the **resource** was not found
- Use parameterized repository methods — never build SQL from request text
- Keep service and repository reusable outside Flask

> "Route handlers should be thin adapters: parse input, call service, serialize output."
>
> — `Advanced/lessons/lecture/Day9_Hour2_Advanced.md` → `### Talk points`

Canonical Route Set

Use **plural nouns** for collections:

Route	Method	Purpose
`/records`	`GET`	List all records
`/records`	`POST`	Create a record
`/records/<id>`	`GET`	Fetch one record
`/records/<id>`	`PUT`	Update one record
`/records/<id>`	`DELETE`	Delete one record

Source Alignment

`Advanced/lessons/lecture/Day9_Hour2_Advanced.md` → `### Talk points`

Demo: GET and POST

```python
from flask import request, jsonify

@app.get("/records")
def list_records():
    records = service.list_records()
    return jsonify({"data": [r.to_dict() for r in records]})

@app.post("/records")
def create_record():
    payload = request.get_json(silent=True)
    if payload is None:
        return error_response("bad_json", "Request body must be JSON", 400)
    try:
        record = service.create_record(payload)
    except ValidationError as exc:
        return error_response("validation_error", str(exc), 400)
    return jsonify({"data": record.to_dict()}), 201
```

Source Alignment

Demo: GET by ID, PUT, DELETE

```python
@app.get("/records/<int:record_id>")
def get_record(record_id: int):
    try:
        return jsonify({"data": service.get_record(record_id).to_dict()})
    except NotFoundError:
        return error_response("not_found", f"Record {record_id} was not found", 404)

@app.put("/records/<int:record_id>")
def update_record(record_id: int):
    payload = request.get_json(silent=True)
    if payload is None:
        return error_response("bad_json", "Request body must be JSON", 400)
    try:
        record = service.update_record(record_id, payload)
    except ValidationError as exc:
        return error_response("validation_error", str(exc), 400)
    except NotFoundError:
        return error_response("not_found", f"Record {record_id} was not found", 404)
    return jsonify({"data": record.to_dict()})

@app.delete("/records/<int:record_id>")
def delete_record(record_id: int):
    try:
        service.delete_record(record_id)
    except NotFoundError:
        return error_response("not_found", f"Record {record_id} was not found", 404)
    return jsonify({"data": {"deleted_id": record_id}})
```

Status Code Decision Guide

Situation	Status
Success — returning existing data	`200`
Success — resource just created	`201`
Request body is not valid JSON	`400`
Field missing or value invalid	`400`
ID refers to a record that does not exist	`404`
Unexpected exception in server code	`500`

Rule: `400` = the caller's fault. `404` = resource not there. `500` = server's fault.

Lab: CRUD API (Hour 34)

Time: 25–35 minutes

Tasks

1. Implement all five routes: list, create, get-by-id, update, delete
2. Test list, create, get by id, update, delete — happy paths
3. Test bad JSON, validation failure, and missing id — sad paths
4. Ensure the service/repository layer is never imported inside a route body

Completion Criteria

- All five routes respond with JSON
- POST returns `201`; missing records return `404`; invalid payloads return `400`
- The service/repository layer remains reusable outside Flask

Common Pitfalls — Hour 34

- **One route for every operation** with manual method branching instead of `@app.get`, `@app.post`
- **Inconsistent integer/string ids** — use `<int:record_id>` in Flask to avoid silent mismatches
- **Returning `200` for a delete that failed** — map the error correctly
- **SQLite exceptions leaking as HTML `500`** — catch domain errors in the route

Source Alignment

`Advanced/lessons/lecture/Day9_Hour2_Advanced.md` → `### Common pitfalls to watch for`

Quick Check — Hour 34

Exit questions:

- When should an API return `400` instead of `404`? Give one tracker example of each.
- What happy path did you prove this hour?
- What sad path did you test or plan to test next?
- Can the same service method be called by a CLI without importing Flask?

Hour 35 — Serialization + Validation

Source Alignment

`Advanced/lessons/lecture/Day9_Hour3_Advanced.md` → `# Day 9, Hour 3`

Hour 35 Learning Outcomes

By the end of this hour, learners will be able to:

- Standardize request and response shapes for the API
- Centralize validation so routes do not duplicate domain rules
- Create predictable JSON errors that clients and tests can rely on

Source Alignment

`Advanced/lessons/lecture/Day9_Hour3_Advanced.md` → `## Learning Outcomes`

Serialization and Deserialization

Serialization — converting Python objects to plain JSON-ready dicts

Deserialization — reading client payloads and turning them into validated data

Key principles:

- The JSON contract is part of the product; changing field names randomly breaks clients
- Manual validation is acceptable when it is **centralized and consistent**
- Do not duplicate validation logic in the GUI, the API, and the repository
- JSON has no `datetime` object — convert dates to ISO 8601 strings explicitly

> "The contract is part of the product."
>
> — `Advanced/lessons/lecture/Day9_Hour3_Advanced.md` → `### Talk points`

Demo: parse_record_payload

```python
from tracker.exceptions import ValidationError

REQUIRED_FIELDS = {"title", "category"}
ALLOWED_FIELDS = REQUIRED_FIELDS | {"status"}
ALLOWED_STATUSES = {"open", "in_progress", "done"}

def parse_record_payload(payload: dict) -> dict:
    if not isinstance(payload, dict):
        raise ValidationError("Payload must be a JSON object")
    unknown = payload.keys() - ALLOWED_FIELDS
    if unknown:
        raise ValidationError(f"Unknown field: {sorted(unknown)[0]}")
    missing = REQUIRED_FIELDS - payload.keys()
    if missing:
        raise ValidationError(f"Missing required field: {sorted(missing)[0]}")
    if not isinstance(payload["title"], str):
        raise ValidationError("title must be a string")
    if not isinstance(payload["category"], str):
        raise ValidationError("category must be a string")
    if "status" in payload and not isinstance(payload["status"], str):
        raise ValidationError("status must be a string")
    title = payload["title"].strip()
    category = payload["category"].strip()
    status = payload.get("status", "open").strip().lower()
    if not title:
        raise ValidationError("title cannot be blank")
    if not category:
        raise ValidationError("category cannot be blank")
    if status not in ALLOWED_STATUSES:
        raise ValidationError(f"status must be one of {sorted(ALLOWED_STATUSES)}")
    return {"title": title, "category": category, "status": status}
```

Demo: record_to_response

```
def record_to_response(record) -> dict:
    return record.to_dict()
```

Why a named helper instead of inline `.to_dict()`?

- One place to rename or add fields without touching every route
- Makes the serialization contract explicit and testable
- Prevents `record.__dict__` leaking private fields or internal state

Anti-pattern to avoid

```
# Do NOT do this — exposes internals, no stable contract
return jsonify(record.__dict__), 200
```

Source Alignment

Validation Sad-Path Matrix

Bad input type	Example	Expected error code
Non-JSON body	Raw string body	`bad_json`
Missing required field	No `title` key	`validation_error`
Wrong type	`"title": 42`	`validation_error`
Unknown field	`"colour": "red"`	`validation_error`
Blank required field	`"title": "  "`	`validation_error`
Invalid enum value	`"status": "maybe"`	`validation_error`

Rule: Validation errors must never expose a Python stack trace.

Benefits of Centralized Validation

Before centralization — validation scattered across routes:

```
# POST /records
if "title" not in payload:
    return error_response("validation_error", "title required", 400)

# PUT /records/<id>
if "title" not in payload:
    return error_response("validation_error", "title required", 400)
```

After centralization — routes become shorter and safer:

```
# POST /records
data = parse_record_payload(payload)   # one call covers all rules
record = service.create_record(data)
```

Source Alignment

Lab: Clean API Contracts (Hour 35)

Time: 25–35 minutes

Tasks

1. Create `api/serializers.py` with `parse_record_payload` and `record_to_response`
2. Route every POST and PUT payload through `parse_record_payload`
3. Manually test three negative cases: missing field, bad type, unknown field
4. Verify that routes become shorter after the refactor

Completion Criteria

- Every endpoint uses the same response wrapper and error wrapper
- Validation messages are helpful without exposing stack traces
- Routes are shorter because parsing and serialization are reusable

Common Pitfalls — Hour 35

- **Different error formats per endpoint** — inconsistency breaks client integrations
- **Duplicating validation** in GUI, API layer, and repository
- **Accepting unknown fields silently** — they hide typos and future field conflicts
- **Forgetting JSON has no `datetime`** — convert dates to strings explicitly

Optional Extensions

- Include `request_id` in all responses, not only errors
- Document the contract in README with one sample request and response
- Add `pytest` tests for `parse_record_payload` without starting Flask

Source Alignment

`Advanced/lessons/lecture/Day9_Hour3_Advanced.md` → `### Common pitfalls to watch for`

Quick Check — Hour 35

Exit questions:

- Why should all API errors follow a consistent format even when the messages differ?
- What happy path did you prove this hour?
- What sad path did you test or plan to test next?
- Which fields are part of the public API versus internal implementation details?

Hour 36 — App Structure: Blueprints and Dependency Wiring

Source Alignment

`Advanced/lessons/lecture/Day9_Hour4_Advanced.md` → `# Day 9, Hour 4`

Hour 36 Learning Outcomes

By the end of this hour, learners will be able to:

- Refactor Flask code into a maintainable application structure
- Use an application factory to wire configuration, repository, and service dependencies
- Avoid circular imports and hidden global state

Source Alignment

`Advanced/lessons/lecture/Day9_Hour4_Advanced.md` → `## Learning Outcomes`

Why Structure Matters

Structure exists to reduce friction, not to impress anyone.

Problems with a single-file approach:

- Can't create an isolated app instance for tests
- DB connection created at import time — no flexibility
- Circular imports appear as the app grows
- Configuration is invisible and hard to change

> "The application factory pattern lets tests create isolated apps with temporary databases."
>
> — `Advanced/lessons/lecture/Day9_Hour4_Advanced.md` → `### Talk points`

Application Factory Pattern

Before — module-level app creation, hidden dependencies:

```
# api/app.py — problematic
from tracker.repository import SQLiteTrackerRepository

repo = SQLiteTrackerRepository("data/tracker.db")   # import-time side effect
service = TrackerService(repo=repo)
app = Flask(__name__)
```

After — explicit factory, wiring visible in one place:

```
def create_app(db_path: Path | None = None) -> Flask:
    app = Flask(__name__)
    database_path = db_path or Path("data/tracker.db")
    database_path.parent.mkdir(parents=True, exist_ok=True)
    repo = SQLiteTrackerRepository(str(database_path))
    repo.init_db()
    service = TrackerService(repo=repo)
    register_routes(app, service)
```

Demo: create_app (api/app.py)

```python
from pathlib import Path
from flask import Flask

from api.routes import register_routes
from tracker.repository import SQLiteTrackerRepository
from tracker.service import TrackerService


def create_app(db_path: Path | None = None) -> Flask:
    app = Flask(__name__)
    database_path = db_path or Path("data/tracker.db")
    database_path.parent.mkdir(parents=True, exist_ok=True)
    repo = SQLiteTrackerRepository(str(database_path))
    repo.init_db()
    service = TrackerService(repo=repo)
    register_routes(app, service)
    return app
```

Source Alignment

Demo: register_routes (api/routes.py)

```python
from flask import Flask
from tracker.service import TrackerService


def register_routes(app: Flask, service: TrackerService) -> None:
    @app.get("/health")
    def health():
        return {"status": "ok"}

    @app.get("/records")
    def list_records():
        return {"data": [r.to_dict() for r in service.list_records()]}

    # ... POST, GET/<id>, PUT/<id>, DELETE/<id> here
```

Key design decisions:

- `service` is injected — no global import needed inside route functions

Circular Import Anti-Pattern

The trap — routes importing app while app imports routes:

```
api/app.py   imports   api/routes.py
api/routes.py   imports   api/app.py   ← circular!
```

Prevention rules:

- `api/routes.py` must never import `api/app.py`
- Pass `service` as a parameter, not as a module-level global
- Never create repositories or open connections at import time
- Use `create_app()` as the composition root — everything wired there

Source Alignment

`Advanced/lessons/lecture/Day9_Hour4_Advanced.md` → `### Common pitfalls to watch for`

Lab: Refactor API Structure (Hour 36)

Time: 25–35 minutes

Tasks

1. Create `api/routes.py` with `register_routes(app, service)` helper
2. Create `api/app.py` with `create_app(db_path=None)` factory
3. Construct repository and service inside `create_app`, not at module level
4. Run `/health` and at least one CRUD endpoint to confirm the refactor works

Completion Criteria

- App still starts after the refactor
- `/health` and at least one CRUD endpoint still work
- DB path can be changed for tests or demos via `create_app(db_path=...)`

Common Pitfalls — Hour 36

- **Circular imports** between `app.py` and `routes.py`
- **Creating the database connection at import time** — prevents test isolation
- **Moving files without updating imports** — always run after moving, not just after writing
- **Refactoring and changing behavior at the same time** — refactor first, then extend

Optional Extensions

- Use a Flask Blueprint for records routes
- Add a config object or environment variable for the database path
- Add a test that creates an app with `tmp_path` from pytest

Source Alignment

`Advanced/lessons/lecture/Day0_Hour4_Advanced.md` → `### Common pitfalls to watch`

Quick Check — Hour 36

Exit questions:

- What risk appears when a repository or database connection is created at import time?
- What happy path did you prove this hour?
- What sad path did you test or plan to test next?
- Where is the one place a reader can see how the app is assembled?

Capstone Quality Gate — Session 9

Before leaving, check each artifact against this rubric:

Category	Ready	Needs Attention
Correctness	Feature works with a realistic happy path	Only works with hard-coded data
Error handling	Sad path produces a clear JSON error	App crashes or returns an HTML traceback
Structure	Responsibility is in the correct layer	Route, service, and SQL logic are tangled
Repeatability	Command or test reproduces the result	Result depends on manual memory
Maintainability	Names are clear, next feature has an	Works once but hard to

Session 9 Summary

What we built today

- `GET /health` — first proof the server is live
- Five CRUD routes backed by `TrackerService` and `SQLiteTrackerRepository`
- `parse_record_payload` and `record_to_response` — centralized contract helpers
- `create_app(db_path)` — application factory with explicit dependency wiring

Design principles reinforced

- Routes are thin adapters, not business logic containers
- One JSON error envelope for every sad path
- No import-time side effects; configuration injected at construction time
- A thin vertical slice per hour, always runnable before extending

What's Next — Day 10

Session 10 builds on today's foundations:

- Testing the Flask API with `pytest` and Flask `test_client()`
- Writing focused integration tests with an in-memory or temp-file database
- Packaging the tracker for distribution and repeatable installation
- Final capstone review and delivery preparation

Commit message to write before Day 10:

```
feat: add Flask API with health, CRUD endpoints, and application factory
```